

Friend Group Expense App Product Plan and Architecture

Build the first version as a mobile first React Native application centered on one unusually clear workflow: a member records a shared expense with a required event name and event date, the app calculates who owes whom, the debtor marks the payment as sent with optional proof, and the recipient confirms that the money arrived. Every signed in member gets a private My plan view and can create their own groups. Voice dictation, receipt OCR, and photos explicitly shared from WhatsApp should enter the same reviewed expense form without a transcription key, WhatsApp API, or media processing server. The product should track payments made elsewhere and must not hold money in the MVP.

The product should not compete by copying every feature in Splitwise or tricount. The closest current competitor already supports browser joining, nudges, mark as paid, and recipient confirmation. The opportunity is a more focused verification inbox, safer proof handling, and a friend group personality layer that makes reminders warm without becoming harassment.^{1,4,8,9}

Recommended product direction

- **Positioning** Shared expenses that close only when both people agree the payment landed.
- **MVP platform** A TypeScript React Native frontend in frontend and a TypeScript Node.js API in backend, connected through a versioned HTTPS contract.
- **Account model** Name and verified email only, using a passwordless magic link or email code.
- **Personal workspace** My plan is the default member specific dashboard; every member may create and participate in multiple groups.
- **Primary wedge** A settlement review queue with proof, confirmation, rejection, audit history, and reminders that automatically pause when a payment is under review.
- **Smart entry** On device speech, receipt OCR, and share sheet photo intake that suggest field values but never save financial data without review.
- **Frontend system** NativeWind supplies Tailwind style utilities over shared design tokens, with exactly one base Button, Input, and Spinner component.
- **Default visual theme** Dusk Blend, with a charcoal summary band, warm off white working surfaces, mint actions, coral debt states, and amber humor accents.
- **Working name** OweYaar, pending trademark, domain, social handle, and app store clearance.

Contents

1. Research findings and competitive benchmark
2. Product strategy and intended users
3. MVP scope and core journeys
4. Information architecture and screen design
5. On device voice transcription and OCR
6. WhatsApp and native share import
7. Visual system, component library, theme, and splash
8. Settlement state model
9. Technical architecture and data model
10. Email reminders and friend group tone

11. Privacy security and abuse prevention
12. Delivery roadmap and success metrics
13. Naming recommendations and risk decisions

Research findings and competitive benchmark

Current products prove that group expense tracking is a well understood behavior. Splitwise offers broad split methods, groups, external settlements, recurring expenses, and receipt scanning in its paid tier. Tricount emphasizes shared entry, offline use, uneven splits, photos, multiple currencies, and payment requests. Settle Up and Splid compete on understandable balances, reduced transaction counts, offline use, and low sign up friction.^{1,2,3,4,5,6,7}

The proposed verification loop is not unique by itself. KittyPot publicly describes a five step sequence that ends with the debtor marking a payment as paid and the recipient confirming that it landed. It also supports browser joining, links, codes, QR invitations, receipts, and nudges. Bexbox presents proof backed IOUs and partial payments. These products validate the need but narrow the differentiation available.^{8,9}

Product	Strength	Relevant features	Opening	Design lesson	Sources
Splitwise	Breadth and mature balance tracking	Groups, flexible splits, external settlement, recurring expenses	Feature density and paid gating leave room for a calmer core flow	Use as a completeness benchmark, not a screen template	1 2 3
tricount	Fast group collaboration	Link invites, shared entry, offline use, photos, currencies, payment requests	Bank request availability is regional	Copy its plain language and fast group entry	4 5
Settle Up	Clear settlement suggestions	Sync across a group and fewer suggested transfers	Public material emphasizes accounting over verification	Make the final who pays whom answer equally obvious	6
Splid	Very low friction	No sign up, offline mode, more than 150 currencies, export	Lower identity assurance is a poor fit for proof confirmation	Preserve its speed while verifying email ownership	7
KittyPot	Closest workflow match	Browser join, link code and QR invite, receipts, nudges, mark paid and recipient confirms	The proposed idea needs a sharper reason to switch	Differentiate through verification inbox, proof privacy, and email personality	8
Bexbox	Evidence first ledger	Proof attachments, partial payments, private ledgers, link and QR invites	Narrow and Android focused public positioning	Treat payment evidence as sensitive content	9

What the market evidence changes

- Do not describe bilateral payment confirmation as a first in the category.
- Make the settlement inbox the fastest and best explained part of the product, because that is where the proposed app can feel meaningfully different.
- Keep the first version small. Mature competitors already cover currencies, charts, exports, and payment integrations; matching that breadth would delay learning. Include voice and OCR only as assisted entry, not as an automatic bookkeeping system.

- Remove app download as a condition for invite acceptance. Current competitors show that browser based joining is expected.
- Avoid a public trust score. The useful outcome is a shared record, while a reputation score can turn friendly debt into social punishment.

Product strategy and intended users

Product promise

The product keeps a shared record of who paid, who owes a share, the name and calendar date of the event, what the share covers, and whether repayment has been confirmed. It removes repeated calculation and gives both sides a clear closing action. It does not replace a bank, wallet, or cash payment method.

Initial users

User	Current difficulty	Required outcome
Friend group organizer	Pays for bookings or meals and currently chases people in chat	Add an expense in under one minute and see every open settlement
Debtor	Wants the exact amount and an easy way to show payment	Open one link, review the share, tap I paid, and attach proof
Expense recipient	Needs to know which transfers arrived	Review a compact queue and confirm or flag each payment
Occasional guest	Does not want another installed app	Join from a secure browser link and verify email

Jobs the MVP should solve

- When I cover a group cost, give the event a recognizable name and date, record it before I forget, and divide it without doing arithmetic in chat.
- When I open the app, show my own plan first: what I owe, what others owe me, which payments need my review, and the groups I recently used.
- When I want a new circle, let me create a group myself, invite friends, and move between my groups without asking a platform administrator.
- When a receipt photo is already in WhatsApp, share that one photo into the app and continue from a prepared expense draft instead of downloading and uploading it again.
- When I owe money, show exactly why, where to pay outside the app, and what remains after a partial payment.
- When I send money, let me mark the payment as sent and provide evidence without exposing that evidence to the entire group.
- When someone says they paid me, let me confirm receipt or explain the problem without changing the ledger incorrectly.
- When a balance stays open, remind the right person at a reasonable cadence and stop as soon as the state changes.

Positioning boundaries

The app should use words such as track, record, claimed, and confirmed. It should not say that it processed, transferred, secured, or guaranteed a payment. Every settlement screen should state that money moves outside the app.

The original expense payer is the recipient or creditor during repayment. The member who owes the share is the debtor. Interface labels should be written from the viewer's perspective: I paid for the debtor and Money received for the recipient. This avoids the ambiguous word payer.

MVP scope and core journeys

MVP capability list

Area	Included behavior
Identity	Name, verified email, magic link or six digit email code, profile theme, private My plan dashboard
Groups	Every member can create multiple groups, becoming that group's owner; base currency, group accent, roles, invite link, invite email, revoke invite
Expenses	Required event name and event date, amount, optional merchant or note, one expense payer, equal, exact value, or percentage split, optional receipt, edit history
Assisted entry	Field dictation, receipt OCR, camera capture, and one image received from the native share sheet, with mandatory review
Balances	Per group and overall amounts, who owes whom, expense level explanation, no hidden simplification
Settlements	Full or partial amount, I paid, optional proof, awaiting review, confirm, needs attention, reversal
Notifications	Invite, new expense, reminder, proof submitted, confirmed, needs attention, bounce handling
Controls	Reminder cadence, quiet hours, mute group, tone preference, data export, account deletion

Defer until after product validation

- **Conversational expense parsing** Do not promise that one spoken sentence can reliably identify every payer, member, amount, and split across every language. Start with field level dictation and reviewed suggestions.
- **Downloaded OCR language packs** Add Tesseract based packs only for launch languages that the native OCR engines do not cover and only after device testing.
- **Multiple currencies** Use one currency per group at launch. Add conversion only with a dated exchange rate and an explicit source.
- **Recurring expenses** Useful for flatmates, but not required to test the verified settlement loop.
- **Payment app links** Add region specific deep links after selecting a launch country and checking platform rules.
- **Desktop application** Keep the web surface focused on invitations and essential review until the native mobile workflow is validated.
- **Debt simplification** Keep off by default because redirected debts are harder to explain and verify.

Explicitly out of scope

The MVP should not hold funds, connect bank accounts, provide credit, arbitrate disputes, expose public payment feeds, calculate a social credit score, or let a member send unlimited reminders. These features raise compliance, security, or relationship risk without proving the main value.

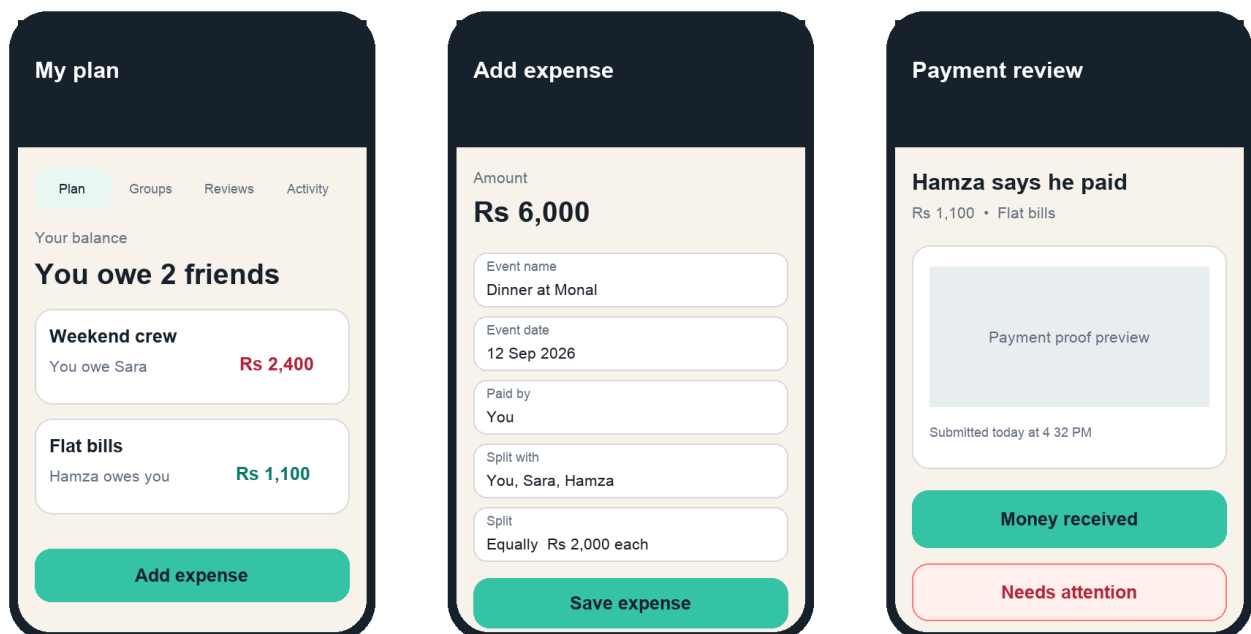
Core journey

1. **Create or join** A member enters an email address, uses a one time link or code, adds a display name, and either creates a group or enters an invited group.
2. **Record** The expense payer types, dictates, scans a receipt, or shares one photo from WhatsApp. The member confirms a required Event name and Event date, then chooses the amount, group, payer, participants, and split method before saving. On device tools may suggest the merchant, printed date, and total.
3. **Review** Every affected member receives a concise notice and can open the expense explanation or raise an issue.
4. **Settle** The debtor pays elsewhere, returns to the app, taps I paid, enters the amount, and optionally uploads proof.
5. **Confirm** The recipient reviews the settlement and taps Money received or Needs attention.
6. **Close** The balance changes only after confirmation. Both members receive a short success message and the audit trail remains visible.

Information architecture and screen design

Dusk Blend mobile concept

A dark summary band over warm light working surfaces



Original concept showing the member specific My plan dashboard, top tabs, expense entry, and payment review

Personal plan and group ownership

My plan is the default signed in view for every account. It is not a public plan or a manually maintained checklist: it is a personalized financial action summary derived from the authenticated member's groups, expenses, and settlements. It shows You owe, Owed to you, Awaiting your review, upcoming reminder state, and recent groups. A person sees only records allowed by active memberships and bilateral proof rules.

Every verified member can create multiple groups. The creator becomes that group's owner and may invite members, assign another admin, change group presentation settings, archive the group, or leave after transferring ownership. Creating one group never gives the member access to another group's data. Group roles control administration; ordinary financial actions remain limited by who paid, who owes, and who receives a settlement.

Navigation

Use a sticky top tab switcher immediately below the signed in header: My plan, Groups, Reviews, and Activity. The selected tab keeps its state when the member opens an item and returns. Show small numeric badges only for actionable reviews, not for ordinary history. Keep profile, theme, and notification settings behind the member avatar. A single persistent Add expense action appears on My plan and Groups. Do not duplicate these destinations in a bottom bar; on compact screens the top tabs scroll horizontally while preserving full labels and accessible roles. Expo Router supplies universal routes and deep links, while React Navigation's top tab integration supports swiping, scrolling tabs, indicators, and badges behind the single TopTabs wrapper.^{15,16}

Top tab	Content	Behavior
My plan	Personal balances, due or open items, recent groups, and next actions	Default after sign in; scoped to the current member
Groups	Groups the member owns or has joined, plus Create group	Any verified member can create a group
Reviews	Payment claims awaiting this member's confirmation and items needing attention	Badge counts only decisions the member can make
Activity	Expenses, edits, reminders, claims, confirmations, and reversals	Filters persist during the current session

Screen	Primary content	Acceptance standard
Sign in	Email field, Continue button, code or link confirmation, name only for first use	A new member reaches the invited group without creating a password
My plan	You owe, Owed to you, Awaiting your review, recent groups, primary Add expense action	The default view answers what this member needs to do before showing history
Groups	Owned and joined groups, Create group, group balance summaries, invite state	Every verified member can create a group and open any group they actively belong to
Group detail	Group balance, settle suggestion, members, chronological ledger, invite	A member can explain any number by opening its expenses

Screen	Primary content	Acceptance standard
Add expense	Event name, event date, image preview, amount, microphone, Scan receipt, merchant or note, paid by, participants, split method	Typed, dictated, scanned, and shared photo entry all end in the same reviewable form
Import expense	Source image, discard, keep as receipt, group, event name, event date, OCR suggestions, Continue	A WhatsApp share opens a draft without saving or uploading anything automatically
Settle	Recipient, amount, external payment note, I paid, optional proof	Payment evidence is not required when friends trust each other
Payment review	Amount, sender, group, proof, timestamp, Money received, Needs attention	No unrelated actions compete with the decision
Reviews	Payment claims awaiting a decision and items needing attention	Only actionable items for the signed in member receive a badge
Activity	Expenses, edits, reminders, claims, confirmations, reversals	Financial history is readable, attributable, and filterable
You	Name, email, theme, tone, reminders, quiet hours, exports, deletion	People control their own inbox and theme

Event identity fields

Field	Member experience	Validation and meaning
Event name	Required short title such as Birthday dinner or Murree weekend	Editable text, 1 to 80 visible characters; never use a receipt filename as the title
Event date	Required calendar date on which the meal, trip, booking, or shared cost occurred	Default to today for manual entry; OCR may suggest a printed date, but the member must confirm it
Merchant or note	Optional supporting detail such as Monal or Advance tickets	Keep separate from the event name so reminders remain understandable
System timestamps	Created at and updated at record application activity	Do not replace the event date when an expense is imported late or edited later

Store Event date as a date only value interpreted in the group's timezone, separate from created at and updated at timestamps. Show Event name and Event date together on expense details, activity items, settlement reviews, and reminder emails so friends can recognize what the balance refers to.

Interaction rules

- Show the amount before every confirmation action and use the group currency symbol plus ISO code when ambiguity is possible.
- Keep destructive or corrective actions behind a More menu, but never hide the dispute action on an open settlement.
- Use optimistic updates only for reversible local actions. A confirmed settlement should wait for a committed server response.
- Never rely on green and red alone. Pair every state color with a label and icon.

- Show the calculation behind every share, including rounding, in one tap.
- When a proof is submitted, replace reminder actions with Awaiting review and stop scheduled nudges immediately.
- Treat voice, OCR, and shared image output as suggestions. Highlight every inserted field and require the member to review the final event name, event date, amount, payer, people, and split before saving.

On device voice transcription and OCR

The app can provide useful multilingual dictation and OCR without an application owned transcription service or API key. The React Native layer does not perform recognition itself; it bridges the operating system or a bundled native engine. These engines use machine learning internally, so the accurate product promise is on device processing with no cloud AI account, not no AI. Language and offline availability vary by device, operating system, installed models, and script.^{26,27,29,30}

Voice dictation design

Use expo speech recognition in an Expo development build. It wraps Apple Speech on iOS and Android SpeechRecognizer behind one React Native interface and exposes on device capability and locale checks. Do not start with the archived React Native Voice package. Native libraries require a development build because Expo Go cannot contain arbitrary native code.^{28,31,32}

Concern	MVP behavior	Guardrail
Language choice	Default to the app language but let the member choose a BCP 47 locale before recording	Never label the feature as every language; display only locales reported by the device
Offline gate	Check on device support and installed locale assets before activating the microphone	If unavailable, offer model download where supported or return to keyboard entry
Recording	Short field level dictation for event name, note, merchant, date, or amount	Do not upload or retain audio; stop immediately when the member leaves the screen
Review	Show partial text while listening, then place the final transcript in an editable preview	Apply only after the member taps Use text
Parsing	Use locale aware number and date parsing plus exact group member name matching	No free form command may silently choose participants or save a split

On iOS, require on device recognition only when SFSpeechRecognizer reports that the selected locale supports it. On Android, check isOnDeviceRecognitionAvailable and the supported locale list, then use the on device recognizer. The ordinary Android recognizer may stream audio to a provider, so strict local mode must not silently fall back to it. Automatic language detection is device dependent and should not replace a visible language selector.^{26,27,28}

Receipt OCR design

Capture a still image with React Native Vision Camera, then run OCR after the member crops and rotates the receipt. Avoid continuous frame processing in the MVP. On Android, ML Kit Text Recognition v2 can run from a bundled model or a model downloaded through Google Play services. It supports Latin, Chinese, Devanagari, Japanese, and Korean scripts. On iOS, Apple Vision performs text recognition on device and can report the languages supported by the current request revision.^{29,30,35}

Stage	MVP behavior	Privacy and accuracy rule
Capture	Edge guide, focus warning, flash control, crop and rotate	Keep the original local until the member decides whether to attach it
Recognize	Return text blocks and positions on device	Do not send the image to an OCR server
Extract	Use deterministic patterns for totals, printed event dates, tax, and currency; use a detected merchant only as a starting point for Event name	A rule can suggest a value but cannot replace user confirmation
Review	Overlay detected candidates and let the member tap the correct total and merchant	Show the source crop beside each suggestion
Persist	Save confirmed structured fields; upload the receipt only when the member explicitly keeps it	Delete temporary images and raw OCR text after the flow completes

Language coverage strategy

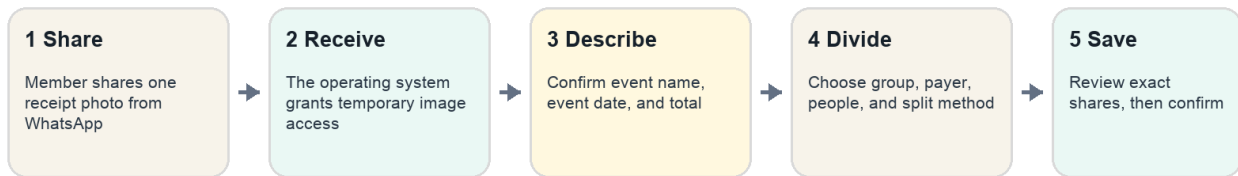
Ship a tested language matrix rather than a universal claim. The first release should support every locale that the phone reports as available for on device dictation and the scripts supported by the selected native OCR engine. For important launch scripts outside that set, a later native Tesseract module can use downloadable or bundled language packs. Tesseract provides data for more than 130 languages and over 35 scripts, including Urdu, but accuracy, binary size, right to left layout, and receipt formatting must be tested per language. Tesseract.js is not the React Native solution because its own documentation says React Native lacks the required WebAssembly support.^{33,34}

Tier	Implementation	Product promise
Tier 1	Device native speech and OCR	Fastest path, lowest app size, best privacy; exact coverage varies by device
Tier 2	Optional native Tesseract language pack	Broader OCR scripts such as Urdu; larger downloads and more QA
Fallback	Keyboard, camera attachment, and manual amount entry	Always available when recognition is unsupported or uncertain

WhatsApp and native share import

Treat WhatsApp as an example source application, not as a data integration. A member opens a receipt photo in WhatsApp, taps the operating system Share action, chooses OweYaar, and continues in an Import expense screen. The app receives only that selected image through the iOS share extension or Android share intent. It does not request chat access, infer the people in the conversation, read message history, or require a Meta or WhatsApp API key.^{36,37,38}

WhatsApp photo to reviewed expense



Privacy boundary

The app receives only the selected image. It cannot read the WhatsApp chat, its members, or message history. The temporary copy is deleted unless the member keeps the receipt.

A shared WhatsApp photo becomes a local draft and reaches the ledger only after the member verifies every financial field

Import journey

Step	Member experience	Required guardrail
Share	In WhatsApp, open one receipt photo, tap Share, and select OweYaar	The app accepts one JPEG, PNG, or WebP image in the MVP
Receive	Copy the granted image into an application cache entry with a short local expiry	Do not upload, create an expense, or retain the source URI automatically
Unlock	If the app is locked, preserve the local draft while the member verifies email or unlocks the app	Show the image only after local authentication; expire abandoned drafts
Review image	Preview, rotate, crop, replace, discard, or choose Keep as receipt	Warn when the image may contain chat names, phone numbers, bank details, or unrelated text
Event details	Enter a required Event name and Event date; OCR may suggest the merchant as a starting title and the printed receipt date	Both fields remain editable and Save stays disabled until the member confirms them
Choose people	Select a group, Paid by, and the members included in the expense	Default Paid by to Me but never infer chat participants or silently select a group
Divide	Choose Equal, Exact values, or Percentages and inspect the calculated amount for each member	Disable Save until the allocation is mathematically complete
Confirm	Review Event name, Event date, total, payer, people, and exact shares, then tap Save expense	Only confirmed structured data reaches the backend; upload the image only if Keep as receipt is selected

Split methods

Method	Interaction	Validation and rounding
Equal	Select the included members; include the expense payer by default with a visible toggle	Divide integer minor units evenly and assign any leftover units deterministically
Exact values	Enter the amount assigned to every included member and show Remaining at all times	The entered values must be nonnegative and sum exactly to the expense total
Percentages	Enter a percentage for every included member and show the calculated money amount beside it	Percentages must total 100.00 percent; allocate rounding by largest remainder with stable member order as the tie break

Weighted shares such as one share versus two shares and item by item receipt assignment are useful follow ups. They should reuse the same allocation engine, but should not make the launch form harder to understand. Every method must end with a per member preview whose minor units sum exactly to the total.

React Native implementation

Create an IncomingShareAdapter so the expense form does not depend on one package. Expo Sharing now documents inbound shared data and MIME type configuration, but it labels the receive path experimental and notes an iOS behavior that Apple does not officially support. Use it for the technical spike, then keep the production gate explicit: Android receives ACTION_SEND with image MIME types, while iOS uses a dedicated Share extension and App Group handoff if the official Expo path does not pass store and cold start testing.^{36,37,38}

Layer	Implementation	Reliability rule
Android	ACTION_SEND intent filter for image JPEG, PNG, and WebP; read the granted content URI	Copy immediately to cache because the temporary grant may disappear after the activity ends
iOS	Small Share extension accepts one image and places a transient reference in an App Group container	Keep the extension UI minimal and move OCR, group selection, and splitting into the main app
Navigation	Normalize warm start, cold start, and locked app shares into one Import expense route	Consume every payload once and prevent a stale image from reopening after Save or Discard
Local queue	Random import identifier, MIME, byte size, checksum, created time, and expiry	Never synchronize device paths, WhatsApp metadata, or raw OCR text
Unsupported input	Explain that launch supports one receipt image and offer camera or gallery import	Reject multiple images, text, PDFs, video, oversized images, and malformed data in the MVP

Share import acceptance tests

- A single WhatsApp photo opens the same Import expense screen on a recent iPhone and two supported Android devices when the app is closed, backgrounded, and already open.

- Sharing does not create an expense, notify a group, or upload an image before the member taps Save expense.
- Save remains disabled until a nonblank Event name and valid Event date are confirmed, even when OCR fills other fields.
- Discard, successful save without Keep as receipt, timeout, and sign out remove the temporary image and OCR text.
- Equal, exact value, and percentage modes always display allocations that sum to the imported total in integer minor units.
- A forwarded image cannot expose another group, and a member cannot select people outside the chosen group unless they explicitly create and send a new invitation.
- The app behaves clearly when WhatsApp supplies an unreadable image, revokes the temporary URI, or the device has no supported OCR engine.

Visual system component library theme and splash

The default theme should be neither a conventional light theme nor a full dark theme. Dusk Blend uses a dark summary layer for orientation and warm light surfaces for data entry and reading. This gives the app a distinctive mixed appearance while keeping dense financial details easy to scan.

Token	Hex	Role	Text pairing
Ink	17212B	Top bars, summary areas, primary text	White or warm canvas
Warm canvas	F7F3EA	Main working background	Ink
Mint	35C3A5	Primary actions and confirmed status	Ink, not white
Coral	FF7A6B	Amount owed and needs attention accents	Ink or dark red text
Amber	F5C451	Reminder and playful tone accents	Ink
Slate	637083	Secondary text and inactive icons	White or warm canvas after contrast testing

Theme choices

Theme	Availability	Behavior
Dusk Blend	Default	Dark summary band plus warm light content surfaces
Cloud	Manual option	Light surfaces throughout, with Ink navigation and text
Midnight	Manual option	Dark surfaces with softened white text and the same semantic status labels
System	Manual option	Follows the operating system light or dark preference

Theme	Availability	Behavior
Group accent	Per group	Changes decorative accent only; it never redefines debt, warning, or success colors

Reusable React Native component contract

Use NativeWind for Tailwind style class utilities in the React Native frontend and map its Tailwind configuration to semantic design tokens such as surface canvas, surface raised, text primary, action primary, status debt, and status confirmed. Screens may combine approved utilities, but they must not introduce raw hex colors, one off spacing values, or lookalike controls. NativeWind supports Expo and depends on Tailwind configuration plus React Native libraries rather than browser CSS at runtime.³⁹

Primitive	Single source	Supported variants	Reuse rule
Button	frontend/src/components/ui/Button.tsx	primary, secondary, ghost, destructive; small, medium, large; icon positions; loading and disabled	All taps that look like buttons use this export; loading replaces the label safely and prevents a second submission
Input	frontend/src/components/ui/Input.tsx	text, email, decimal, password, search, multiline; label, hint, error, leading or trailing icon	All editable text fields use this export; date pickers and member selectors use the same FieldShell but are not fake text inputs
Spinner	frontend/src/components/ui/Spinner.tsx	small, medium, large; light or dark contrast; inline or centered	One accessible progress indicator for buttons and delayed content; never build a second loader for a screen
TopTabs	frontend/src/components/navigation/TopTabs.tsx	fixed or horizontally scrollable; badge; controlled selection	One keyboard and screen reader aware switcher for My plan, Groups, Reviews, and Activity
FieldShell	frontend/src/components/forms/FieldShell.tsx	label, requirement, hint, error, disabled	Keeps text input, date, member, and split controls aligned without duplicating validation presentation

Feature screens compose these primitives; they do not copy their styles. Add a pull request check that rejects imports from private primitive internals, a component gallery covering every state and theme, and unit or interaction tests for focus, validation, disabled, loading, and reduced motion behavior. The backend has no Tailwind dependency because it renders no interface.

Lightweight animated splash

Show an immediate native Expo splash using the Ink background and a static OweYaar mark, then cross fade into a short React Native handoff animation after the first application frame. The motion should use only logo opacity, a small scale from 0.96 to 1.00, and at most one mint accent translation. Target 650 to 900 milliseconds and enforce a 1.2 second ceiling. Expo recommends hiding the native splash as soon as the application is ready and testing the final behavior in a release build because Expo Go and development builds do not fully reproduce the installed splash experience.⁴⁰

Implement the handoff with React Native Reanimated and animate non layout properties such as transform and opacity. Do not use a video, GIF, particle system, remote asset, or continuous loop. Do not wait for an API response: while the native splash is visible, load only bundled fonts, local theme tokens, and cached session state; fetch My plan data after navigation is interactive. Reanimated recommends non layout properties for smoother updates and can follow the operating system reduced motion setting, in which case the app should use a near instant cross fade.^{41,42}

- Performance budget: no more than two animated elements, no layout animation, no image decoding above the device splash asset size, and no JavaScript timer chain.
- Quality gate: record cold start on a lower end supported Android device and a recent iPhone in release mode; reject visible stutter, blank frames, double splash, or delayed touch readiness.
- Failure behavior: if local preparation exceeds the ceiling, reveal the application shell and use the single Spinner only for the specific content still loading.
- Accessibility: honor Reduce Motion, preserve sufficient logo contrast, and never communicate status through motion alone.

Material Design 3 recommends semantic color roles and design tokens rather than assigning raw colors ad hoc, and it supports dynamic color on compatible Android experiences. The React Native theme layer should map the same semantic tokens to iOS and Android. System and manual themes remain the launch commitment; Android dynamic color can be added as an optional platform enhancement.^{20,21}

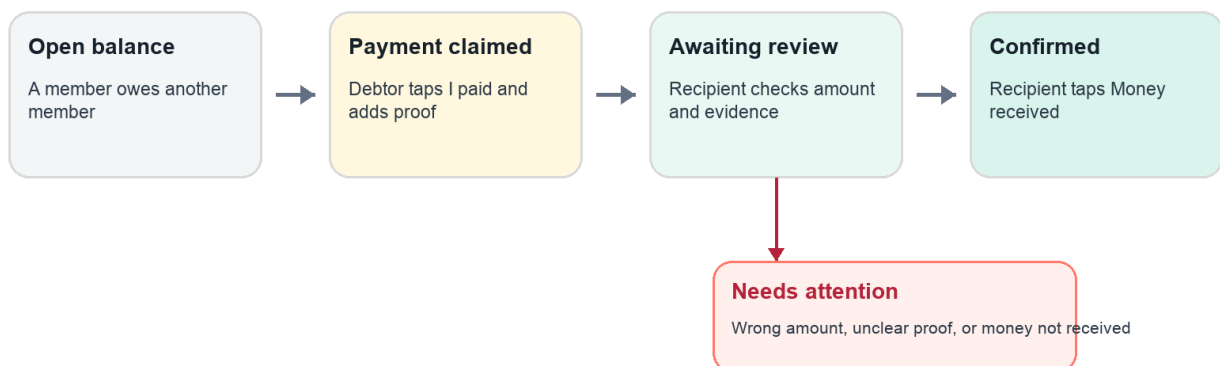
Target WCAG 2.2 AA. Maintain at least 4.5 to 1 contrast for normal text, provide visible keyboard focus, use comfortably sized targets, label all controls, and make authentication usable without memory tests or puzzles. WCAG 2.2 adds guidance for minimum target size and accessible authentication.²⁰

Writing style

Use short direct labels: Add expense, You owe Sara, I paid, Awaiting review, Money received, and Needs attention. Humor belongs in optional email or celebration copy, not in amounts, error messages, disputes, or privacy controls.

Settlement state model

Verified settlement state model



Balances change only after recipient confirmation; a problem path returns the settlement to an actionable state

State definitions

State	Meaning	System behavior
Open	A confirmed expense creates an amount owed	Debtor can start a full or partial settlement
Claimed	Debtor states that money was sent	Amount, time, method note, and optional proof are recorded
Awaiting review	Recipient has not decided	Automatic reminders to the debtor stop; recipient receives a review notification
Confirmed	Recipient says the money arrived	Settlement is posted to the ledger and the remaining balance is recalculated
Needs attention	Recipient reports missing money, wrong amount, or unclear evidence	Settlement does not change the balance; both people can add a note or submit corrected proof
Reversed	A confirmed record was entered incorrectly	A compensating ledger entry restores the amount and preserves the audit history

Integrity rules

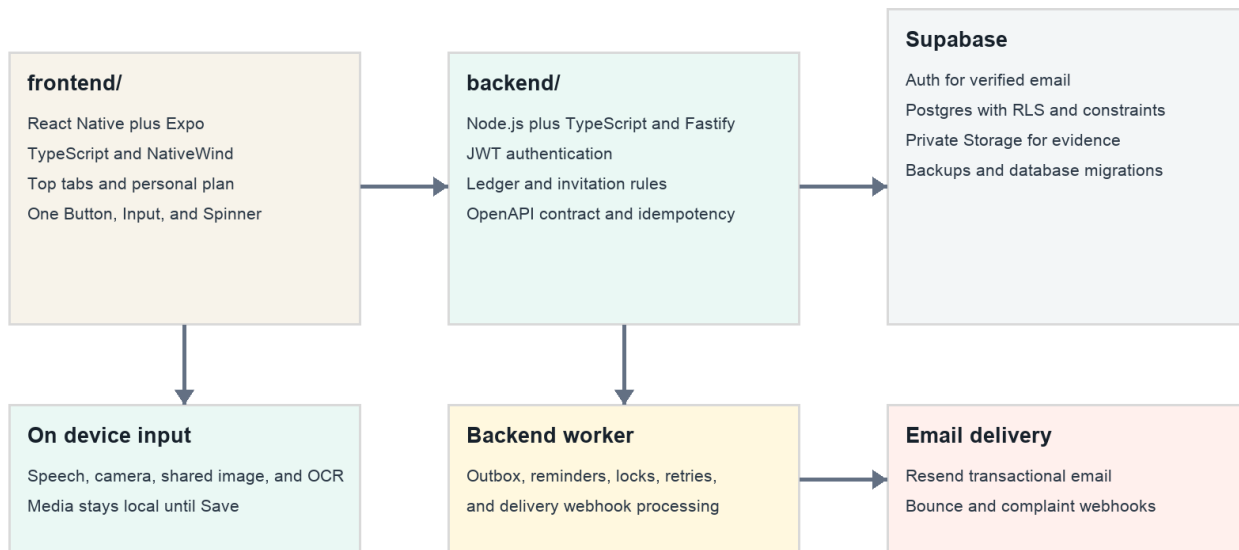
- Only the debtor can create a payment claim for their own balance unless the recipient records a cash payment on their behalf and the activity log names that action.
- Only the intended recipient can confirm or reject the claim.
- Confirmation and reversal endpoints require idempotency keys so double taps and retries cannot post twice.
- A partial settlement cannot exceed the currently open amount without a second explicit confirmation.
- Edits to expenses that already have settlements create an adjustment entry and notify affected members.
- Debt simplification remains disabled at launch. Traceable pairwise balances are easier to explain when evidence and confirmation matter.

Rounding

Store all money as integer minor units with an ISO 4217 currency code. For an equal split, divide the total minor units, assign the same base share to each participant, and distribute remaining units deterministically in participant order. For percentage splits, store percentages as integer basis points, calculate exact fractional shares, and use the largest remainder method with stable participant order as the tie break. Display every allocated amount before save. Never use binary floating point for persisted money.

Technical architecture

Recommended MVP architecture



The app sends a Supabase access token to the Node API; privileged database keys never ship in the frontend

Two application folders create a clear boundary: the React Native frontend calls the Node.js backend, which applies business rules before using Supabase and the email provider

Repository and runtime boundary

Start with one repository and exactly two deployable application folders. The separation is architectural from day one, but a single workspace keeps dependency updates, contract generation, and integration tests manageable. Do not add a third shared package at the start; generate the frontend API client from the backend OpenAPI contract into the frontend folder.

Location	Responsibility	Planned contents
frontend/	React Native and Expo application for iOS, Android, and the small invite or authentication web fallback	app routes; src/components/ui; src/components/forms; src/components/navigation; src/features; src/lib/api; src/theme; tests
backend/	Node.js and TypeScript API plus a separately started background worker	src/routes; src/domain; src/services; src/repositories; src/workers; src/lib/supabase; supabase/migrations; tests
Repository root	Workspace coordination only	package manager workspace file, lockfile, lint and format configuration, README, continuous integration

Keep secrets in environment specific configuration. Commit example environment files with names only, never values. The frontend receives the public Supabase URL and publishable key plus the Node API base URL. The backend receives server credentials, database configuration, email credentials, and allowed

application origins. Database migrations live under backend because the Node service owns business persistence.

Recommended stack

Layer	Choice	Reason
Frontend	React Native, TypeScript, Expo development build, Expo Router, React Navigation top tabs, NativeWind	One application folder for iOS and Android plus a small universal invite fallback and shared Tailwind style tokens
UI primitives	One Button, one Input, one Spinner, one TopTabs, and one FieldShell	Variants handle visual states without duplicated screen level components
Voice	expo speech recognition with strict on device capability checks	No transcription API key; audio stays on the phone when local mode is available
Camera and OCR	React Native Vision Camera plus Apple Vision or Google ML Kit through a local Expo module	Native capture and recognition with explicit script and device support
Inbound sharing	IncomingShareAdapter with Expo Sharing spike, Android ACTION SEND, and an iOS Share extension fallback	One reviewed image from WhatsApp or another app without a WhatsApp API or media server
Backend API	Node.js, TypeScript, and Fastify with JSON Schema and OpenAPI	A low overhead business boundary for authorization, validation, settlements, invitations, and signed file access
Background jobs	A worker process from the backend folder	Transactional outbox, reminders, retries, locks, and email webhook processing
Database	Supabase Postgres	Transactions, constraints, audit records, and row level security
Authentication	Supabase email magic link or OTP	Verified email without passwords; redirect URLs must be allowlisted
Files	Private Supabase Storage bucket	Receipt and proof objects protected by membership policies and short lived signed URLs
Email	Resend transactional delivery	Templates, domain authentication, delivery events, bounces, and complaints
Deployment	Containerized Node API and worker near the Supabase project region; local or EAS mobile builds	Provider remains a deployment decision; voice, OCR, and share intake need no media processing server
Operations	Structured logs, error tracking, uptime checks, database backups	Enough observability to investigate money state and email delivery

Expo development builds are required because speech, OCR, camera, and inbound share features include native iOS and Android code. Builds can run locally; EAS is optional. Keep the universal web surface inside frontend and limit it to invite links, authentication callbacks, and essential review rather than duplicating the full native workflow. The native share route is mobile only; it should not be presented as a web feature.^{31,36,39}

Fastify supports TypeScript and request or response validation through JSON Schema, making it suitable for a compact versioned Node API. Publish an OpenAPI document in continuous integration and generate a

typed frontend client so the two folders cannot silently disagree on amounts, dates, split payloads, or settlement states.⁴³

Frontend to backend connection

Step	Contract	Guardrail
1 Authenticate	The frontend signs in with Supabase email magic link or OTP and receives a managed access token	Only the publishable key is present in the shipped application
2 Call API	The generated frontend client sends Authorization Bearer access token, idempotency key for mutations, and a versioned JSON payload to backend v1	TLS, strict origins, request size limits, and request identifiers are required
3 Establish actor	Fastify verifies the Supabase JWT or resolves it with Supabase Auth, then attaches user id, email verification, and request context	Never accept a profile or member id from the body as proof of identity
4 Authorize and commit	Domain services check group membership and settlement roles; repositories write one transaction to Supabase Postgres	Use the user's JWT for RLS protected access where practical; reserve privileged credentials for isolated jobs
5 Respond and refresh	The API returns a documented result and the frontend query cache updates My plan, Groups, Reviews, and Activity	Retries reuse the same idempotency key and never double post a ledger effect

Supabase Auth uses JWTs and accepts them in the Authorization Bearer header. Supabase recommends row level security for frontend accessible data and warns that secret or service role credentials bypass RLS and must never be shipped in a browser or mobile application. For this design, the frontend talks directly to Supabase only for authentication; normal financial reads and writes go through the Node API. The backend should prefer a request scoped Supabase client carrying the member token so RLS remains active, while the outbox worker uses privileged credentials only after selecting jobs through tightly scoped server code.^{10,11,44,45,46}

Notification execution

Use a database outbox instead of sending email inside the expense transaction. The transaction writes the expense, shares, activity event, and pending notification job together. A worker claims jobs, sends through Resend, stores the provider message ID, and marks the job sent. Delivery webhooks then update delivered, delayed, bounced, failed, or complained states.¹⁸

Run the reminder worker as a second process from backend. It must use a database lock and a unique key such as settlement, member, reminder type, and cadence date because scheduled execution and network delivery are at least once behaviors in practice. Failed jobs remain in the outbox with bounded exponential backoff, while idempotency prevents concurrent workers or restarts from sending the same reminder twice.

Data model and application interfaces

Entity	Key fields	Constraint
profiles	id, auth user id, display name, verified email, theme, timezone, preferred locale	One row per account

Entity	Key fields	Constraint
input preferences	profile id, dictation locale, strict local mode, OCR script	Stores settings only, never audio or raw OCR images
groups	id, name, base currency, accent, created by, archived at	Any verified profile may create a group and becomes its initial owner
group members	group id, profile id or invited email, role, state, joined at	Unique active member per group and identity; exactly one or more owners according to transfer rules
invites	group id, email optional, token hash, expires at, max uses, revoked at	Raw tokens are never stored
expenses	group id, event name, event date, amount minor, currency, merchant or note, paid by, created at, updated at, version, source type	Event name and date are required; event date is distinct from system timestamps; one expense payer
expense shares	expense id, member id, owed minor, split method, basis value	Owed minor units sum to the expense; percentage basis points sum to 10000
settlements	group id, from member, to member, amount minor, state, claimed at, confirmed at	Supports partial settlement and one currency
settlement proofs	settlement id, storage key, mime, size, hash, submitted by	Private object metadata only
ledger entries	group id, member pair, amount minor, currency, source type, source id, reversed by	Append only financial effect
activity events	group id, actor, event type, subject, safe metadata, created at	Human readable audit history
notification jobs	recipient, type, payload reference, due at, state, dedupe key, provider id	Reliable email outbox
notification preferences	profile, group optional, channel, cadence, quiet hours, tone, muted	Recipient controls override group defaults

Event name and Event date belong to the expense in the MVP, so entering them does not require a separate event creation screen. A later multi expense trip feature may normalize reusable events into their own table. Incoming share payloads remain in a short lived device cache; the server receives only the final confirmed expense and an image only when Keep as receipt is selected.

Core server interfaces

Interface	Responsibility
GET /v1/me/plan	Return the signed in member's owe, owed, review, and recent group summary
GET and POST /v1/groups	List active memberships or create a group with the actor as owner
POST /v1/groups/:id/invites	Issue an email bound or limited use invite token
POST /v1/invites/accept	Validate token, verify email, and join idempotently

Interface	Responsibility
POST /v1/expenses	Require Event name and Event date, validate shares, and commit expense ledger entries
PATCH /v1/expenses/:id	Create a versioned adjustment and notifications
POST /v1/settlements	Create a full or partial payment claim
POST /v1/settlements/:id/proof	Validate and attach a private image
POST /v1/settlements/:id/confirm	Authorize recipient and post settlement ledger entry
POST /v1/settlements/:id/attention	Pause closure and record a structured reason
POST /v1/notification-preferences	Set personal cadence, tone, mute, and quiet hours

Authorization policy

Every group scoped query must require an active membership, including My plan aggregation. A member may read group ledger data but can read a settlement proof only when they are the settlement sender or recipient. File paths should include opaque IDs, and access should use short lived signed URLs. Supabase secret or legacy service role keys stay server side because they bypass row level security; the mobile app receives only a publishable key.^{11,12,44}

Email reminders and friend group tone

Email is a core workflow channel, not decoration. Every expense or reminder message should identify the group, Event name, Event date, amount, current state, and one safe action. Do not attach or preview payment proof in email. The message should link to an authenticated in app review page.

Email events

Event	Default timing	Primary action	Guardrail
Invitation	Immediately	Join group	One reminder after 48 hours; expire after seven days
New expense	Immediately or daily digest	Review share	Affected members only
Friendly reminder	Day 3 after amount becomes open	View and settle	Skip if muted, disputed, or awaiting review
Cheeky reminder	Day 7	View and settle	Recipient selected tone controls the wording
Final automatic reminder	Day 14	View and settle	Then weekly only if the recipient explicitly enables it
Payment claimed	Immediately	Review payment	Sent to the recipient, not the group
Payment confirmed	Immediately	View receipt record	Sent to both people
Needs attention	Immediately	Resolve issue	No humor and no automated debtor reminders

Tone controls

- **Friendly** Neutral and warm. This is the default for every account.
- **Cheeky** Light jokes that never mention a member publicly or question their character.
- **Chaos** More playful group language, available only when the recipient opts in.
- **Quiet** Important state changes only, with no automatic reminders.

The group may choose a default tone pack, but each recipient's own setting wins. Manual nudges require a 24 hour cooldown, automatic reminders stop after three by default, and every message includes a group mute link. Disputed and awaiting review states suppress payment reminders.

Sample message library

Moment	Subject	Body direction
Invitation	The group chat did the talking. Now let the numbers behave.	Sara invited you to Weekend Crew. Join to see and add shared expenses.
Friendly reminder	A small reminder from Weekend Crew	You still have Rs 2,400 open with Sara for Dinner at Monal on 12 Sep. If you already paid, mark it and add proof.
Cheeky reminder	Your balance is still doing overtime	Rs 2,400 is waiting for its retirement. Settle it or open the expense if something looks wrong.
Chaos reminder	Breaking news your Rs 2,400 has not left the chat	No panic. Pay outside the app, then tap I paid so Sara can confirm.
Proof submitted	Hamza says the money is on its way	Review the Rs 1,100 payment and confirm only after it appears in your account.
Confirmed	Settled and officially out of the group lore	Sara confirmed Rs 2,400 received. Your balance is now clear.
Needs attention	This payment needs a quick check	Sara could not confirm Rs 1,100. Open the payment to see the reason and respond.

Delivery standards

Send from a dedicated subdomain and configure SPF, DKIM, and DMARC. Resend recommends verifying SPF and DKIM and optionally adding DMARC; Google requires authentication for Gmail delivery and sets additional rules for high volume senders. Store bounce and complaint events, suppress hard bounces, provide visible controls, and keep spam complaints far below Google's 0.3 percent ceiling.^{17,18,19}

Privacy security and abuse prevention

On device input privacy

Strict local speech mode must never fall back to a network recognizer without a separate informed choice. The app should not retain microphone audio. OCR temporary files and raw detected text should remain in the application cache and be deleted after the user applies or discards the suggestions. Analytics may

record success, failure category, duration band, locale, and device capability, but never transcript text, receipt text, or images.

Shared image privacy

A photo shared from WhatsApp may contain chat names, phone numbers, message previews, faces, or unrelated media. Preview it before OCR, make crop and discard actions prominent, and explain that only the selected image was received. Do not store the source application, conversation identity, original device URI, OCR text, or image pixels in analytics. Delete the cache copy after Save or Discard unless the member explicitly chooses Keep as receipt.

Proof handling

Payment screenshots may reveal account numbers, balances, transaction references, notifications, and contact names. Before upload, tell members to crop sensitive details and offer an in app crop and blur step. Strip image metadata, compress the image, and delete proof automatically after a configurable retention period such as 30 days after confirmation unless both people choose to keep it.

- Allow JPEG, PNG, and WebP only for the MVP. Do not accept PDFs or videos until there is a demonstrated need.
- Verify extension, decoded file signature, and MIME type; do not trust the browser Content Type header.
- Generate the stored filename, limit dimensions and file size, re encode images, and scan uploads where practical.
- Use private storage, group and settlement authorization, and short lived signed URLs.
- Do not expose proof in email, activity feeds, analytics payloads, or support logs.

OWASP recommends allowlisting file types, validating signatures and MIME types, generating filenames, setting size limits, authorizing uploaders, storing files outside public web paths, and scanning when available.¹³

Invitation safety

Generate at least 128 bits of random token entropy, store only a cryptographic hash, use HTTPS, bind private invitations to the intended email, expire links, support revocation, and rate limit acceptance. Generic group links should have a short lifetime and maximum use count. OWASP guidance for URL tokens recommends cryptographically random values, secure storage, single use, expiry, trusted redirect destinations, HTTPS, and rate limiting.¹⁴

Group safety

- A member can mute reminders without leaving the group or hiding the balance.
- A user can report abusive copy or a group, and support can suspend reminder sending independently of financial records.
- Names and custom group text require length limits and output encoding. Email templates must escape all user content.
- Group admins can remove future access but cannot erase another member's confirmed financial history silently.
- Export and deletion behavior must be defined before launch. Account deletion should anonymize activity where complete erasure would corrupt other members' ledgers.

Security acceptance tests

Test	Pass condition
Cross group access	A member cannot query another group's expense, proof, invite, or activity row
Proof URL	An expired signed URL fails and a non participant cannot issue another one
Double confirmation	Repeated requests create one ledger effect
Invite replay	Used, expired, revoked, or email mismatched tokens fail safely
Reminder duplication	Concurrent workers send at most one message per cadence window
File disguise	Renamed executable and malformed image inputs are rejected
Shared payload lifetime	Expired or reused share payloads fail safely and a consumed image cannot reopen a second draft
Audit integrity	Corrections create reversals rather than deleting confirmed history

Delivery roadmap and success metrics

A focused private beta with on device voice, OCR, and cross platform share import can be delivered in roughly thirteen weeks by one experienced full stack engineer with part time product design and structured QA support. A solo first time builder should expect a longer schedule. The timeline below is a planning assumption, not a vendor estimate.

Timing	Focus	Exit condition
Week 1	Validate	Interview five to eight friend groups; map current chat and spreadsheet behavior; test terminology
Week 2	Prototype	Clickable My plan, top tabs, create group, typed or shared image expense entry, split selection, settle, review, and lightweight splash
Weeks 3 and 4	Foundation	Create frontend and backend folders; NativeWind tokens and shared primitives; Node API contract; Auth, profiles, groups, memberships, invitations, and RLS
Weeks 5 and 6	Ledger	Personal plan query, expenses, exact rounding, balances, activity history, edits, and frontend to backend integration tests

Timing	Focus	Exit condition
Weeks 7 and 8	Settlement	Payment claims, private proof, confirmation, attention state, outbox, and emails
Weeks 9 and 10	On device input	Speech capability gate, camera capture, OCR, equal, exact, and percentage split engine, privacy tests
Week 11	Share import	Android share intent, iOS share extension path, cold and warm start routing, local queue, WhatsApp device tests
Week 12	Hardening	Release build splash profiling, reduced motion, component audit, accessibility, security cases, device matrix, email authentication, export and deletion
Week 13	Private beta	Launch to three to five real groups, watch events, collect interviews, and fix blockers

Definition of done for private beta

- A new member can accept an invite and reach the intended group in under two minutes without a password.
- Every verified member can create a group, becomes its owner, can join multiple groups, and sees only authorized group data in My plan.
- The sticky top tabs switch among My plan, Groups, Reviews, and Activity, preserve return state, and expose review badges accessibly without a duplicate bottom navigation system.
- An equal expense with four members saves with exact shares whose minor units sum to the total.
- A debtor can submit a full or partial payment claim and proof; only the recipient can view and decide it.
- The confirmed settlement changes the ledger exactly once, including under retries and double taps.
- Reminder emails stop during review, respect tone and mute choices, and record delivery failures.
- On supported devices, dictation and OCR work without an application API key; unsupported locales fail clearly to manual entry and no media is uploaded.
- On iOS and Android, sharing one WhatsApp receipt photo opens a local draft, requires a confirmed Event name and Event date, supports equal, exact value, and percentage splits, and uploads nothing before confirmation.
- All screens import the single Button, Input, and Spinner primitives; their states render correctly in every theme and no screen contains a visually duplicated private version.
- Frontend business mutations pass through the Node API with a valid Supabase token and idempotency key; expired tokens, unauthorized memberships, and malformed contracts fail predictably.
- The native and React Native splash transition has no blank frame or network dependency, completes within the agreed ceiling, remains smooth on the lower end reference Android device, and honors Reduce Motion.
- All core screens meet the agreed WCAG 2.2 AA checks and work at common mobile widths.

Product metrics

Metric	Definition	Why it matters
Activation	Percentage of invited members who verify email and open the group within 24 hours	Measures invite friction
First value	Median time from group creation to first correctly shared expense	Tests entry speed
Assisted entry completion	Percentage of voice, camera OCR, or shared image attempts that produce a confirmed expense without retyping every field	Measures useful accuracy rather than raw recognition confidence
Share import completion	Percentage of received share images that become a confirmed expense or an intentional discard	Measures reliability and identifies abandoned or confusing imports
Settlement completion	Percentage of payment claims confirmed within seven days	Measures the core outcome
Review time	Median time from I paid to Money received	Measures recipient clarity
Dispute rate	Percentage of claims moved to Needs attention	Surfaces wrong math, unclear proof, or trust issues
Reminder health	Mute rate, bounce rate, complaint rate, and manual nudge frequency	Prevents fun email from becoming spam
Group retention	Groups with a new expense in a later month	Separates trip only use from ongoing value

Recommended launch test

Run the beta with a mix of one trip group, one flat or household group, and one recurring social group. Do not start with a public launch. Observe whether members understand I paid versus Money received, whether proof is actually used, how many reminders are tolerated, and whether the mixed theme remains readable in bright and low light conditions.

Naming recommendations

OweYaar is the strongest working name. It is short, sounds like the familiar expression oh yaar, states the debt concept through owe, and supports both warm and chaotic reminder copy. A preliminary web search did not surface an obvious exact product collision, but this is not trademark, company registry, domain, social handle, or app store clearance.

Name	Role	Why it works	Risk
OweYaar	Recommended	Warm, memorable, directly tied to owing, strong South Asian friend group personality	May feel regional if global expansion is immediate

Name	Role	Why it works	Risk
TabToli	Strong alternate	Tab plus a group of friends; playful and group centered	Meaning of toli may need explanation outside South Asia
PaidNa	Cheeky alternate	Feels like a real reminder between friends	Question like name can feel nagging and is less clear in search
HisabScene	Descriptive alternate	Connects shared accounts with casual plans	Longer and culturally specific
OweHo	Playful alternate	Short, rhythmic, works well in reminder subjects	Less obvious pronunciation and meaning
SceneSet	Brandable alternate	Positive friend group phrase and good celebration language	Does not explain expense tracking on its own

Names to avoid

SettleBro is already used by a 2026 bill splitting app that advertises payment confirmation, reminders, and invite links. PingTab is used by at least a driver logistics app and a browser link sharing product. TabMates is already the name of an open source expense application, and Khatakhat is used by active consumer brands. These are poor launch candidates even before formal legal review.^{22,23,24,25}

Name clearance checklist

1. Search exact and similar marks in the launch country's trademark database and relevant financial software classes.
2. Search Apple App Store, Google Play, GitHub, company registries, and major search engines for exact and phonetic matches.
3. Check the preferred domain, sensible alternatives, and core social handles on the same day.
4. Ask ten target users to hear the name once, spell it, and explain what they expect the app to do.
5. Have counsel clear the final candidate before paying for identity design or publishing stores.

Early voice ideas for OweYaar

- Product descriptor: Shared expenses with proof and confirmation
- Friendly completion line: Settled, yaar
- Invite line: Bring the group. Leave the calculator.
- Reminder label inside the app: Send a nudge
- Confirmation celebration: Balance clear. Friendship untouched.

Risks and decisions before development

Risk	Level	Failure	Mitigation
Weak differentiation	High	Closest competitors already confirm payments	Test the verification inbox and tone controls with prototypes before engineering breadth

Risk	Level	Failure	Mitigation
Reminder fatigue	High	People mute or mark mail as spam	Recipient control, default cap, quiet hours, dispute pause, cooldown, delivery monitoring
Sensitive screenshots	High	Bank details leak to group members or logs	Private bilateral access, crop and blur, metadata stripping, short retention, signed URLs
Language coverage gaps	High	A device lacks an offline speech model or OCR script	Runtime capability checks, tested device matrix, visible language selector, manual fallback
OCR amount or date error	High	A subtotal, previous balance, or printing date is mistaken for the final event data	Rank candidates but require confirmation of Event name, Event date, and total
Share extension reliability	High	A WhatsApp photo is lost on cold start or the iOS handoff breaks after an update	IncomingShareAdapter, local expiry queue, exact once consumption, real device upgrade tests, native iOS fallback
Wrong split selection	High	The imported total is assigned to the wrong group, payer, or people	No inferred chat members, visible group and payer fields, allocation preview, disabled Save until valid
Native module drift	Medium	A React Native or operating system update breaks recognition	Pin versions, use development builds, wrap providers behind local interfaces, test upgrades
Component drift	Medium	Screens copy Button, Input, or Spinner styles and themes become inconsistent	One exported primitive per control, component gallery, lint boundaries, and review checklist
Splash jank	Medium	A heavy animation delays interaction or drops frames on lower end phones	Static native first frame, two non layout animations at most, release profiling, time ceiling, and reduced motion
Contract drift	High	Frontend and backend disagree on money, dates, or settlement states	Versioned JSON schemas, generated client, contract tests, and backward compatible changes
Ledger disagreement	High	Edits or rounding make balances untrustworthy	Integer money, database constraints, calculation preview, audit events, reversal entries
Invite hijacking	Medium	Forwarded or guessed links admit the wrong person	High entropy hashed tokens, email binding, expiry, use caps, revocation, rate limiting
Scope expansion	High	Conversational parsing, payments, currencies, and desktop parity delay launch	Keep the MVP decision log and require beta evidence for every added feature

Risk	Level	Failure	Mitigation
Regional product fit	Medium	Payment methods and language differ by market	Choose one launch country after interviews; keep money movement outside the MVP

Decisions still required

Decision	Why it matters
Launch geography	Currency defaults, wording, payment links, privacy notices, and legal review
Launch languages	Select the first tested speech locales, interface translations, OCR scripts, and any Tesseract packs
Invite policy	Email bound invitations only or controlled generic links for very casual groups
Proof requirement	Optional by default is recommended; groups may request it but should not expose it broadly
Retention period	Recommended starting point is automatic proof deletion 30 days after confirmation
Partial payment	Recommended for MVP because it affects the data model and real payment behavior
Expense approval	Recommended to notify and allow disputes, not require every member to approve every expense
Inbound share library	Run Expo Sharing as a spike, but use a dedicated iOS Share extension if the experimental route fails cold start or store testing
Shared image retention	Delete temporary imports after Save or Discard; upload only when Keep as receipt is selected
Node deployment	Choose a container provider and region only after latency, cost, worker support, backups, and Supabase region are known

Immediate next step

Create a clickable prototype of eight views: invite acceptance, My plan with top tabs, Groups with Create group, Import expense from WhatsApp, Add expense with Event name, Event date, and split selection, I paid, Payment review, and profile theme settings. In parallel, build a short architecture spike with the frontend and backend folders: sign in through Supabase, call GET /v1/me/plan through Fastify, verify RLS, generate the typed API client, and profile the release splash on one recent iPhone and two Android devices. The device spike must also enumerate offline speech locales, scan ten real receipts, verify Event date suggestions, exercise cold and warm WhatsApp sharing, check equal, exact, and percentage rounding, and confirm that disabling connectivity does not upload media. Test the prototype with two friend groups before implementing the full ledger.

Sources

1. Splitwise. [Split expenses with friends](#). Official product page. Accessed September 12 2026.
2. Splitwise Help Center. [How do I use Splitwise](#). Official workflow documentation. Accessed September 12 2026.
3. Splitwise. [Splitwise Pro](#). Official paid feature description. Accessed September 12 2026.
4. tricount. [Expense Tracker App Features](#). Official product features. Accessed September 12 2026.
5. tricount Help Center. [Request links](#). Official request link availability and fee information. Accessed September 12 2026.
6. Settle Up. [Settle Up](#). Official product page. Accessed September 12 2026.
7. Splid. [Split expenses the easy way](#). Official product page. Accessed September 12 2026.
8. KittyPot. [Split the cost Not the friendship](#). Official workflow and feature page. Accessed September 12 2026.
9. Bexbox. [Private personal ledger with proof](#). Official product page. Accessed September 12 2026.
10. Supabase. [Passwordless email logins](#). Official authentication documentation. Accessed September 12 2026.
11. Supabase. [Row Level Security](#). Official authorization documentation. Accessed September 12 2026.
12. Supabase. [Storage Access Control](#). Official private storage access documentation. Accessed September 12 2026.
13. OWASP. [File Upload Cheat Sheet](#). Security guidance. Accessed September 12 2026.
14. OWASP. [Forgot Password Cheat Sheet](#). URL token security guidance applied to invitations and magic links. Accessed September 12 2026.
15. Expo. [Introduction to Expo Router](#). Official universal routing and deep link guidance. Accessed September 12 2026.
16. React Navigation. [Material Top Tabs Navigator](#). Official scrollable top tabs, indicators, badges, and swipe integration guidance. Accessed September 12 2026.
17. Resend. [Managing Domains](#). Official SPF DKIM and DMARC setup guidance. Accessed September 12 2026.
18. Resend. [Event Types](#). Official email delivery event documentation. Accessed September 12 2026.
19. Google. [Email sender guidelines](#). Official Gmail authentication, unsubscribe, and spam rate requirements. Accessed September 12 2026.
20. W3C. [Web Content Accessibility Guidelines 2.2](#). W3C Recommendation, December 12 2024 version.
21. Android Developers. [Create an accessible and personalized theme and brand with Material Design 3](#). Official theming guidance. Accessed September 12 2026.
22. Apple App Store. [SettleBro](#). Current app listing used for preliminary name screening. Accessed September 12 2026.
23. Google Play. [PingTab](#). Current app listing used for preliminary name screening. Accessed September 12 2026.
24. GitHub. [TabMates app](#). Existing open source project name identified during preliminary screening. Accessed September 12 2026.
25. Khatakhat Logistics. [Khatakhat](#). Existing consumer brand identified during preliminary screening. Accessed September 12 2026.
26. Apple Developer. [On device speech recognition support](#). Official iOS capability documentation. Accessed September 12 2026.
27. Android Developers. [SpeechRecognizer](#). Official Android on device recognition and capability API documentation. Accessed September 12 2026.
28. GitHub. [Expo Speech Recognition](#). React Native bridge for Apple Speech and Android SpeechRecognizer. Accessed September 12 2026.
29. Google for Developers. [ML Kit Text Recognition v2](#). Official on device OCR capabilities and supported scripts. Accessed September 12 2026.
30. Apple Developer. [Recognizing text in images](#). Official Apple Vision on device text recognition guidance. Accessed September 12 2026.

31. **Expo.** [Add custom native code](#). Official development build and local Expo module guidance. Accessed September 12 2026.
32. **GitHub.** [React Native Voice](#). Archived package notice and migration recommendation. Accessed September 12 2026.
33. **Tesseract OCR.** [Installing Tesseract from Git](#). Official language and script pack coverage documentation. Accessed September 12 2026.
34. **GitHub.** [Tesseract.js framework support](#). Official note that React Native is not supported by the WebAssembly implementation. Accessed September 12 2026.
35. **GitHub.** [React Native Vision Camera](#). Current React Native camera library and release history. Accessed September 12 2026.
36. **Expo.** [Sharing](#). Official send and receive sharing documentation, MIME configuration, and iOS experimental limitation. Accessed September 12 2026.
37. **Android Developers.** [Receive simple data from other apps](#). Official ACTION SEND intent and MIME type guidance. Accessed September 12 2026.
38. **Apple Developer.** [Share extensions](#). Official Share extension purpose, input validation, and host application behavior. Accessed September 12 2026.
39. **NativeWind.** [Installation](#). Official Expo and Tailwind configuration guidance. Accessed September 12 2026.
40. **Expo.** [SplashScreen](#). Official native splash control and release build testing guidance. Accessed September 12 2026.
41. **Software Mansion.** [React Native Reanimated performance](#). Official animation performance guidance, including preference for non layout properties. Accessed September 12 2026.
42. **Software Mansion.** [useReducedMotion](#). Official reduced motion hook documentation. Accessed September 12 2026.
43. **Fastify.** [TypeScript](#). Official TypeScript and JSON Schema guidance for the Node.js framework. Accessed September 12 2026.
44. **Supabase.** [Securing your data](#). Official frontend key, RLS, and backend secret handling guidance. Accessed September 12 2026.
45. **Supabase.** [JSON Web Tokens](#). Official bearer token and JWT verification guidance. Accessed September 12 2026.
46. **Supabase.** [Choosing a server package](#). Official guidance for header based server authentication. Accessed September 12 2026.